

SOMA 멘티교육자료

AWS 클라우드 실습가이드 - 서버리스 DynamoDB

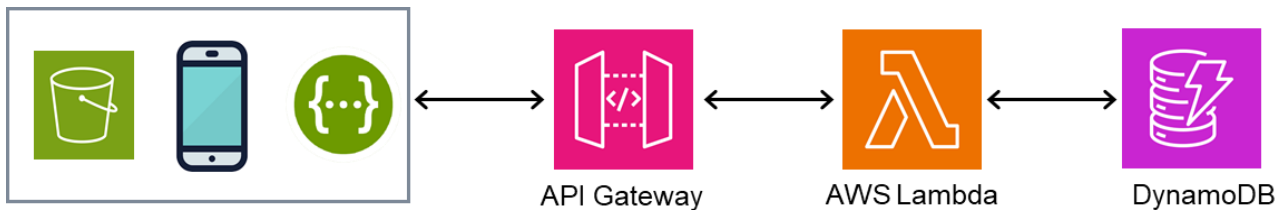
정철웅 멘토

[cwjung123@gmail.com]

서버리스 DynamoDB API 연동

전화번호부를 등록, 수정, 조회, 삭제하는 프로그램 (Serverless 기반: Lambda + DynamoDB + API Gateway)

■ 목표시스템 구성



■ 사전준비 사항

- AWS 계정생성
- AWS IAM 사용자 및 접근키생성
- AWS CLI 설정

■ 설치도구

- NoSQL Workbench for DynamoDB
<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/workbench.html>

■ 활용사이트

- CRUD 예제
<https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-dynamo-db.html>

- API Test
<https://www.postman.com>
<https://swagger.io>
<https://resttesttest.com>
Advanced REST client (<https://install.advancedrestclient.com/>)

■ 기타

- 영어 (English US)기반으로 화면캡처 진행 (일부 혼용)
- Lambda 에 사용된 실소스 및 교육자료 GitHub 링크 강의중 제공
- 강의중 생성된 자료는 첨부 3. 삭제편 확인후 삭제가능

[1 단계. DynamoDB 테이블 생성]

[방법 1. 콘솔]

▶ 콘솔 URL

<https://ap-northeast-2.console.aws.amazon.com/dynamodbv2/home?region=ap-northeast-2#tables>

▶ 테이블생성 --> 데이터입력

테이블명 : phonebook, 파티션키 : id	항목추가 : DynamoDB > 항목탐색 > phonebook																
DynamoDB > Tables > Create table Create table Table details info DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table. Table name This will be used to identify your table. phonebook Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.). Partition key The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability. id String ▼ 1 to 255 characters and case sensitive. Sort key - optional You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key. Enter the sort key name String ▼ 1 to 255 characters and case sensitive. Cancel Create table	id, name, phonenum DynamoDB > Items: phonebook > Item editor Create item Form JSON Attributes Add new attribute ▼ <table><thead><tr><th>Attribute name</th><th>Value</th><th>Type</th><th></th></tr></thead><tbody><tr><td>id - Partition key</td><td>1005</td><td>String</td><td></td></tr><tr><td>name</td><td>Tommy Kim</td><td>String</td><td>Remove</td></tr><tr><td>phonenum</td><td>02050003100</td><td>String</td><td>Remove</td></tr></tbody></table> Cancel Create item	Attribute name	Value	Type		id - Partition key	1005	String		name	Tommy Kim	String	Remove	phonenum	02050003100	String	Remove
Attribute name	Value	Type															
id - Partition key	1005	String															
name	Tommy Kim	String	Remove														
phonenum	02050003100	String	Remove														

[방법 2. CLI] : DynamoDB 이용 권한을 가진 IAM 키 적용된 상태에서 진행

- CLI 테스트 : `aws dynamodb list-tables` (테이블 목록출력)
- 테이블명: phonebook (파티션키)
- 속성 json 파일 작성 (`create-table-phonebook.json`) (파일위치 `~/apistart/reference/dynamo`)

```
{
  "TableName": "phonebook",
  "KeySchema": [
    { "AttributeName": "id", "KeyType": "HASH" }
  ],
  "AttributeDefinitions": [
    { "AttributeName": "id", "AttributeType": "S" }
  ],
  "BillingMode": "PAY_PER_REQUEST"
}
```

- 테이블 생성 CLI

```
aws dynamodb create-table --cli-input-json file://create-table-phonebook.json
```

- 테이블 삭제 CLI

```
aws dynamodb delete-table --table-name phonebook
```

- 데이터등록 1 (partiQL 실행)

aws dynamodb batch-execute-statement --statements file://item_partiql.json

비고. window는 인코딩 ansi로 되어야 함

aws dynamodb batch-execute-statement --statements file://item_partiql_ansi.json

===== item_partiql.json내용 =====

```
[
  {
    "Statement": "INSERT INTO phonebook VALUE {'id' : '1011', 'name' :
'오나라', 'phonenum' : '01010101111' }"
  },
  ----- 중간 생략 -----
  {
    "Statement": "INSERT INTO phonebook VALUE {'id' : '1015', 'name' :
'한나라', 'phonenum' : '01030301111' }"
  }
]
```

- 데이터등록2 (put item방식) --> 간편함

aws dynamodb batch-write-item --request-items file://item_list.json

비고. window는 인코딩 ansi로 되어야 함

aws dynamodb batch-write-item --request-items file://item_list_ansi.json

===== item_list.json내용 =====

```
{
  "phonebook": [
    {
      "PutRequest": {
        "Item": { "id": {"S": "1031"}, "name": {"S": "강감찬"},
        "phonenum": {"S": "01022226565"}
      }
    },
    ----- 이하 생략 -----
  ]
}
```

- DynamoDB 콘솔에서 데이터 확인

Items returned (10)				↺ Actions ▼ Create item	
				< 1 > ⚙️ 🔗	
☐	id (String) ▼	name ▼	phonenum ▼		
☐	1031	강감찬	01022226565		
☐	1012	측나라	01020201111		
☐	1013	한나라	01030301111		
☐	1011	오나라	01010101111		

[2 단계-A. Lambda 함수생성 - 콘솔]

▶ 콘솔 URL

<https://ap-northeast-2.console.aws.amazon.com/lambda/home?region=ap-northeast-2#/functions>

▶ 함수생성

[Node 선택]

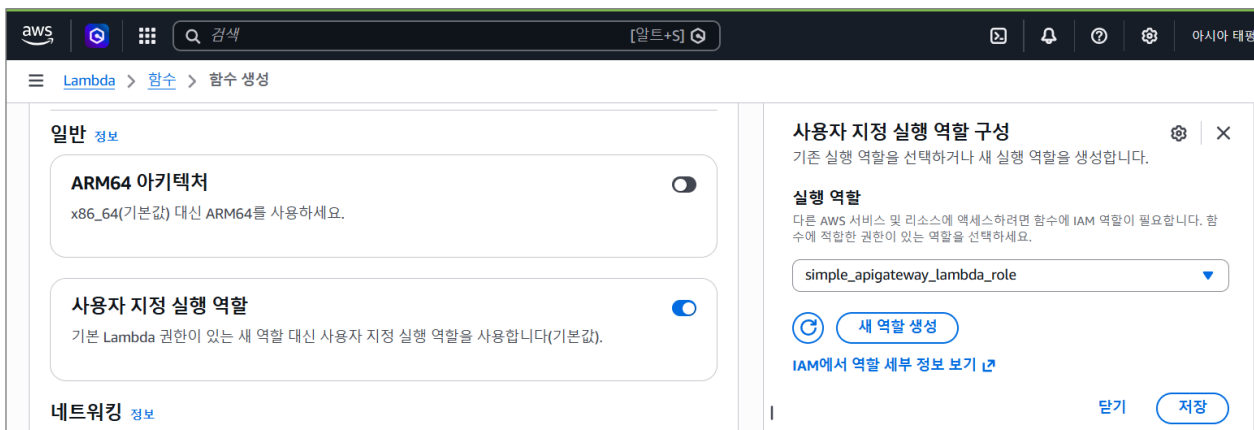
함수명 : test_phonebook_node 런타임 : Node.js 24.x ~

[Python선택]

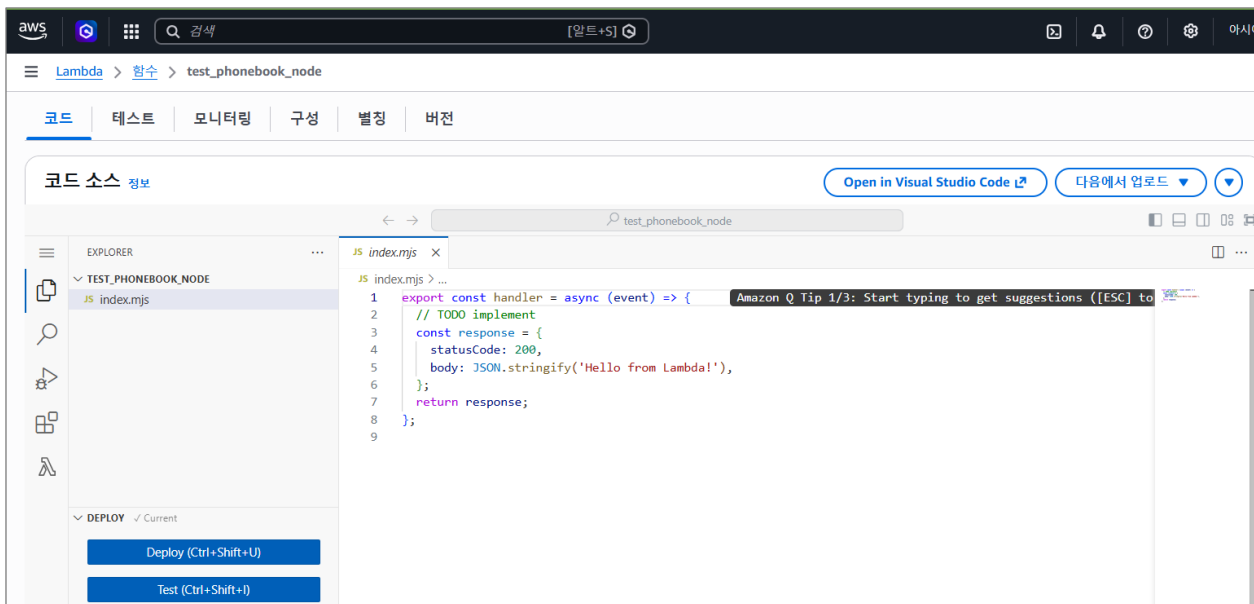
함수명 : test_phonebook_python 런타임 : Pythn3.14

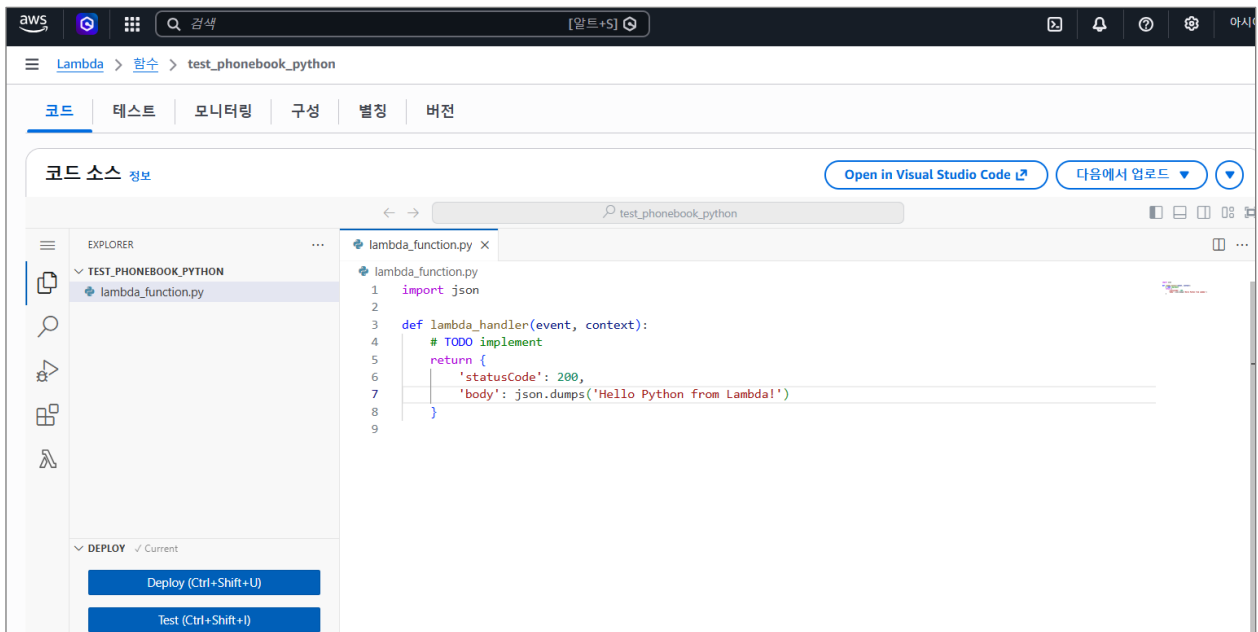
▶ 역할설정

기본 람다 역할을 선택해도 되나 기존에 생성한 역할로 대체해서 진행

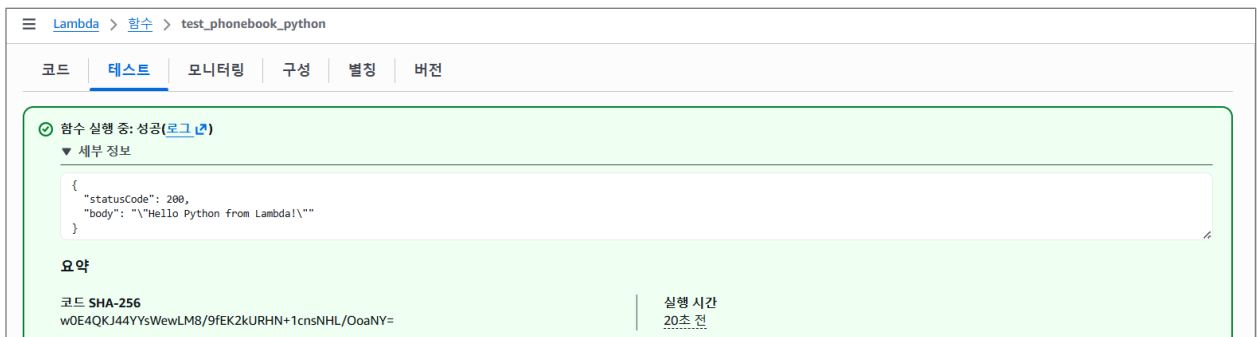
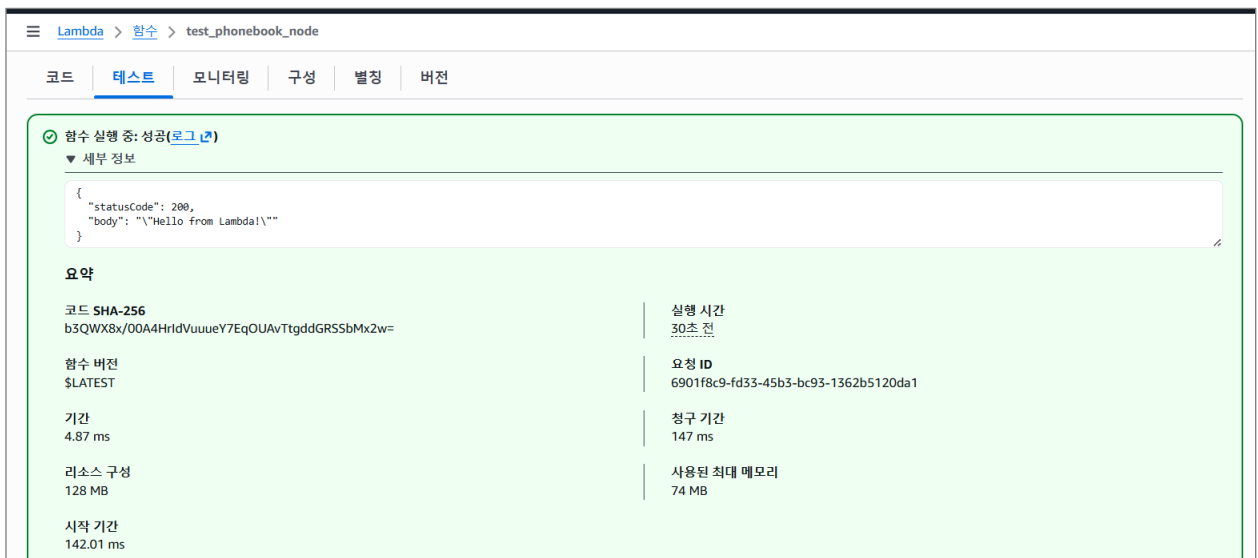


▶ 함수생성 완료화면





▶ 테스트 수행



[2 단계-B. Lambda 함수생성 - CLI]

1. NodeJS 22.X (SDK v3) 기준

- 소스경로로 이동 (ex : ~/apistart/reference/lambda/phonebook_node/phonebook2_clean)
- 모든파일과 디렉토리 압축

```
zip myfunction.zip -r *
```

- Lambda CLI 동작

소스압축본 myfunction.zip(명칭임의) 생성된 것 확인후에 진행

함수명 : phonebook2_node

```
aws lambda create-function --function-name phonebook2_node_function --zip-file  
fileb://myfunction.zip --handler index.handler --architectures arm64 --runtime  
nodejs22.x --role arn:aws:iam::111122223333:role/simple_apigateway_lambda_role
```

2. Python V3.13

- 소스경로로 이동 (ex : ~/apistart/reference/lambda/phonebook_python)

```
cd ~/apistart/reference/lambda/phonebook_python
```

- 모든파일과 디렉토리 압축

```
zip myfunction.zip -r *
```

- Lambda CLI 동작 (myfunction.zip 생성된 것 확인후에) 함수명 : phonebook2_python_function

```
aws lambda create-function --function-name phonebook2_python_function --zip-file  
fileb://myfunction.zip --handler lambda_function.lambda_handler --architectures  
arm64 --runtime python3.13 --role  
arn:aws:iam::111122223333:role/simple_apigateway_lambda_role
```

※ 함수 내용변경

```
aws lambda update-function-code --function-name functionName --zip-file  
fileb://depymentFile.zip
```

※ 함수 역할변경

```
aws lambda update-function-configuration --function-name functionName --role  
arn:aws:iam::111122223333:role/roleName
```


[3 단계. API 생성]

===== A. 생성 =====

순서 :

(Old) API-Gateway생성 --> 경로(route)생성 --> 통합(Integration)생성 --> 통합붙이기 --> CORS설정

(New) API 구성 --> 경로(route)구성 --> 스테이지(stage) 정의 --> 검토 및 생성 --> CORS설정

▶ 유형선택

HTTP-API 는 비용과 성능측면에서 유리

- HTTP-APIs (effective REST APIs) : \$1/1M
- REST APIs : \$3.5/1M
- HTTP-APIs are 14~16% faster than REST APIs

▶ API생성 (명칭 : phonebook_api / 통합은 나중에 진행)

API Gateway > API > API 생성 > HTTP API 생성

1단계
● API 구성

2단계 - 선택 사항
● 경로 구성

3단계 - 선택 사항
● 스테이지 정의

4단계
● 검토 및 생성

API 구성

API 세부 정보

API 이름
HTTP API에는 이름이 있어야 합니다. 이름은 API를 식별하고 구성하는 데 사용하는 고유하지 않은 값입니다. 이 API를 프로그래밍 방식으로 참조하려면 API Gateway에서 자동으로 생성한 API ID를 사용하세요.

IP 주소 유형 | 정보
API의 기본 엔드포인트를 간접적으로 호출할 수 있는 IP 주소 유형을 선택합니다. 업데이트를 적용하기 위해 API를 재배포할 필요는 없습니다.

☒ IPv4
IPv4 주소만 포함합니다.

☐ Dualstack
IPv4 및 IPv6 주소를 포함합니다.

통합 (0) 정보
API가 통신할 백엔드 서비스를 지정하세요. 이러한 서비스는 통합이라고 합니다. Lambda 통합의 경우, API Gateway가 Lambda 함수를 호출하고 이 함수의 응답으로 응답합니다. HTTP 통합의 경우, API Gateway가 지정된 URL로 요청을 보내서 URL의 응답을 반환합니다.

통합 추가

취소

검토 및 생성

다음

(다음 선택)

경로구성

API Gateway > API > API 생성 > HTTP API 생성

1단계
● API 구성

2단계 - 선택 사항
● 경로 구성

3단계 - 선택 사항
● 스테이지 정의

4단계
● 검토 및 생성

경로 구성 - 선택 사항

경로 구성 정보

API Gateway는 경로를 사용하여 API 사용자에게 통합을 표시합니다. HTTP API에 대한 경로는 HTTP 메서드와 리소스 경로(예: GET /pets)의 두 부분으로 구성됩니다. 통합에 대한 특정 HTTP 메서드(GET, POST, PUT, PATCH, HEAD, OPTIONS 및 DELETE)를 정의하거나 ANY 메서드를 사용하여 지정된 리소스에서 정의하지 않은 모든 메서드를 일치시킬 수 있습니다.

경로 추가

취소

검토 및 생성

이전

다음

(다음 선택)

스태이지 정의

API Gateway > API > API 생성 > HTTP API 생성

1단계 API 구성
2단계 - 선택 사항 경로 구성
3단계 - 선택 사항 스테이지 정의
4단계 검토 및 생성

스태이지 정의 - 선택 사항

스테이지 구성 정보

스테이지는 API를 배포할 수 있는 독립적으로 구성 가능한 환경입니다. 스테이지에 자동 배포가 구성된 경우를 제외하고, API 구성 변경을 사용하려면 스테이지에 API를 배포해야 합니다. 기본적으로 콘솔을 통해 생성된 모든 HTTP API에는 \$default라는 이름의 기본 스테이지가 있습니다. API에서 변경한 모든 내용은 해당 스테이지로 자동 배포됩니다. '개발' 또는 '프로덕션'과 같은 환경을 나타내는 스테이지를 추가할 수 있습니다.

스테이지 이름

\$default

자동 배포

☒

제거

스테이지 추가

취소

이전

다음

(다음 선택)

API Gateway > API > API 생성 > HTTP API 생성

1단계 API 구성
2단계 - 선택 사항 경로 구성
3단계 - 선택 사항 스테이지 정의
4단계 검토 및 생성

검토 및 생성

API 이름 및 통합

API 이름

phonebook_api

IP 주소 유형

IPv4

통합

구성된 통합이 없습니다.

편집

경로

경로

구성된 경로가 없습니다.

편집

스테이지

스테이지

- \$default (Auto-deploy: enabled)

편집

취소

이전

생성

[생성완료]

API Gateway > API > 경로 - phonebook_api (wt8cpn5ux9)

API Gateway

API

사용자 지정 도메인 이름

도메인 이름 액세스 연결

VPC 링크

API: phonebook_api ... (wt8cpn5ux9)

Successfully created API phonebook_api (wt8cpn5ux9).

경로

phonebook_api 에 대한 경로

생성

Q 검색

경로를 선택합니다.

스테이지: -

배포

===== B. 경로구성 =====

- GET /items : 전체가져오기

The screenshot shows the '경로 생성' (Create Path) page in the API Gateway console. The breadcrumb navigation is 'API Gateway > API > 경로 - phonebook_api (wt8cpn5ux9) > 경로 생성'. On the left sidebar, under 'API Gateway', there are links for 'API', '사용자 지정 도메인 이름', '도메인 이름 액세스 연결', and 'VPC 링크'. Below these, it says 'API: phonebook_api ... (wt8cpn5ux9)'. The main area is titled '경로 생성' and contains a section '경로 및 메서드 정보' (Path and Method Information). It says '경로 이름 예: /매완 동물' (Path name example: /maewan dongmul) and '메서드를 선택하고 경로를 생성할 경로를 입력합니다. API당 하나의 "\$default" 경로를 지정할 수도 있습니다. API에 대한 요청이 다른 경로와 일치하지 않으면 "\$default" 경로가 호출됩니다.' (Select a method and enter the path to create the path. You can also specify one "\$default" path per API. If a request for the API does not match any other path, the "\$default" path is called). There is a dropdown menu set to 'GET' and a text input field containing '/items'. At the bottom right, there are two buttons: '취소' (Cancel) and '생성' (Create).

- GET /items/{id} : 단항목가져오기

This screenshot is similar to the previous one but shows a green success message at the top: '성공적으로 created 라우팅했습니다 GET /items.' (Successfully created routing for GET /items.). The path configuration is now 'GET' and '/items/{id}'. The '생성' (Create) button is still visible at the bottom right.

- PUT /items : 자료등록/교체

The screenshot shows the '경로 생성' (Create Path) page with the method dropdown set to 'PUT' and the path input field containing '/items'. The '생성' (Create) button is at the bottom right.

- POST /items : 자료등록 (나중에 활용을 위해 생성)

The screenshot shows the '경로 생성' (Create Path) page with the method dropdown set to 'POST' and the path input field containing '/items'. The '생성' (Create) button is at the bottom right.

- DELETE /items/{id}

API Gateway > API > 경로 - phonebook_api (wt8cpn5ux9) > 경로 생성

API Gateway

API

사용자 지정 도메인 이름

도메인 이름 액세스 연결

VPC 링크

API: phonebook_api ... (wt8cpn5ux9)

경로 생성

경로 및 메서드 정보

경로 이름 예: /애완 동물

메서드를 선택하고 경로를 생성할 경로를 입력합니다. API당 하나의 "\$default" 경로를 지정할 수도 있습니다. API에 대한 요청이 다른 경로와 일치하지 않으면 "\$default" 경로가 호출됩니다.

DELETE /items/{id}

취소 생성

[최종 경로설정이 완료된 화면]

API Gateway > API > 경로 - phonebook_api (wt8cpn5ux9)

성공적으로 created 라우팅했습니다DELETE /items/{id}.

API Gateway

API

사용자 지정 도메인 이름

도메인 이름 액세스 연결

VPC 링크

API: phonebook_api ... (wt8cpn5ux9)

▼ Develop

Routes

Authorization

Integrations

CORS

Reimport

Export

경로

phonebook_api 에 대한 경로

생성

검색

▼ /items

PUT

POST

GET

▼ /{id}

DELETE

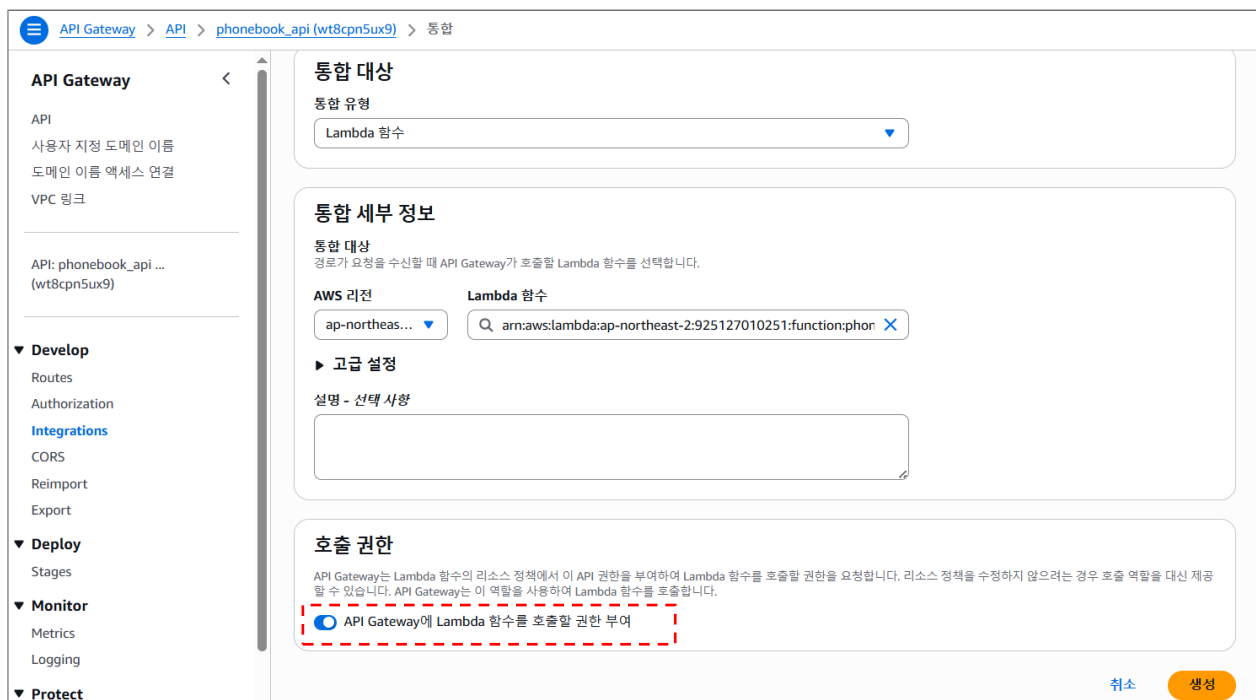
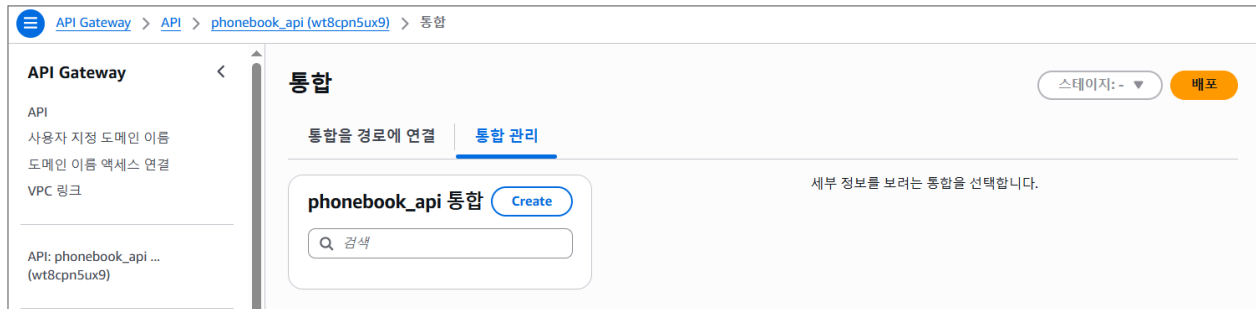
GET

스테이지: - 배포

경로를 선택합니다.

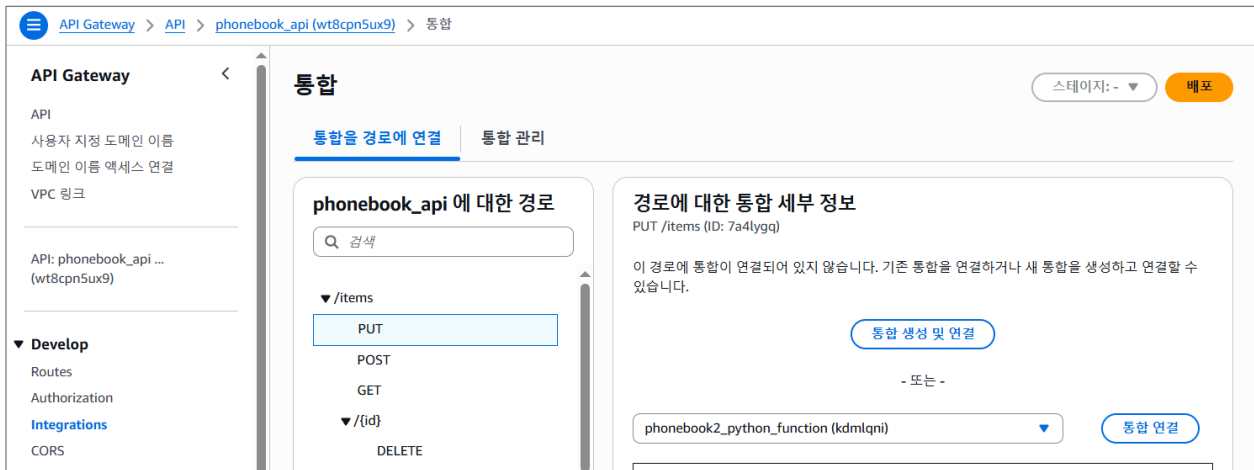
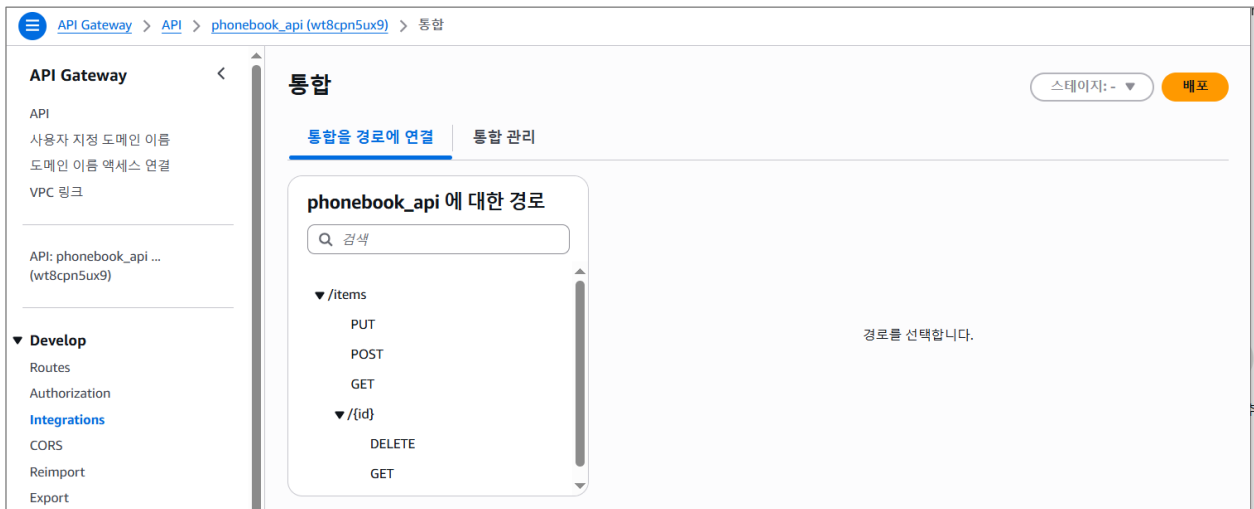
===== C. 통합구성 & 경로설정 =====

▶ 통합생성



- 이 통합을 경로에 연결 : 선택하지 않음(다음 화면)
- API Gateway에 Lambda 함수를 호출할 권한 부여 체크된 상태(기본) --> 생성

▶ 통합을 경로에 연결



위에서 생성한 통합을 각 경로별로 연결

[최종 통합 경로설정 완료화면]



===== D. CORS설정 =====

※ CORS(Cross-Origin Resource Sharing, 교차 출처 리소스 공유)는 브라우저가 서로 다른 도메인의 리소스를 로드할 수 있도록 함.

아래와 3값에 대한 세부처리 혹은 * 입력처리 (*는 개발/테스트환경에서 사용)

Access-Control-Allow-Origin / Access-Control-Allow-Methods / Access-Control-Allow-Headers

(항목별 상세의미)

항목	의미	일반 권장값
Access-Control-Allow-Origin	어떤 프론트 도메인에서 호출을 허용할지	개발 중: * / 운영: https://app.mydomain.com
Access-Control-Allow-Headers	브라우저가 보낼 수 있는 요청 헤더	Content-Type, Authorization, X-Amz-Date, X-Api-Key, X-Amz-Security-Token
Access-Control-Allow-Methods	허용할 HTTP 메서드	OPTIONS, GET, POST
Access-Control-Expose-Headers	브라우저 JS 에서 읽을 수 있게 노출할 응답 헤더	보통 비워둠
Access-Control-Max-Age	Preflight 결과 캐시 시간	300(5 분) 또는 3600 0 : 매번 OPTIONS 요청 발생
Access-Control-Allow-Credentials	쿠키/인증정보 포함 요청 허용 여부	보통 아니요

===== E. 호출테스트 =====

▶ API EndPoint 확인

API Gateway > API > phonebook_api (wt8cpn5ux9)

API 세부 정보

API ID: wt8cpn5ux9
프로토콜: HTTP
설명: -
ARN: arn:aws:apigateway:ap-northeast-2::apis/wt8cpn5ux9

생성됨: 2026-05-17
기본 엔드포인트: <https://wt8cpn5ux9.execute-api.ap-northeast-2.amazonaws.com> (활성화됨)

phonebook_api 에 대한 스테이지 (1)

스테이지 이름	URL 호출	연결된 배포	자동 배포	마지막 업데이트
\$default	https://wt8cpn5ux9.execute-api.ap-northeast-2.amazonaws.com	nuo0j	enabled	2026-05-17

붉은색 박스 URL이 API EndPoint주소임

예시) <https://dl1s0lzvyh.execute-api.ap-northeast-2.amazonaws.com>

▶ 호출방법선택

방법1) EC2 혹은 노트북에서 curl로 진행

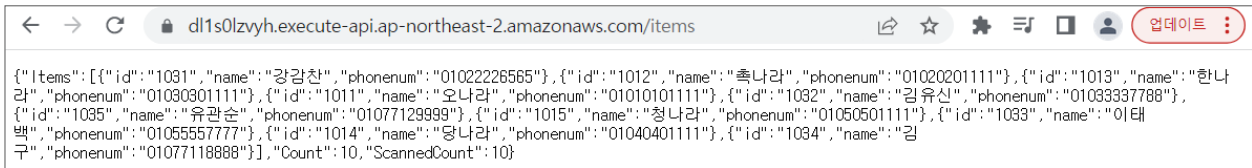
방법2) 웹호출 <https://reqbin.com/curl> (웹브라우저 secret mode)

방법3) Postman 등 API호출 도구사용

▶ API-호출테스트 진행

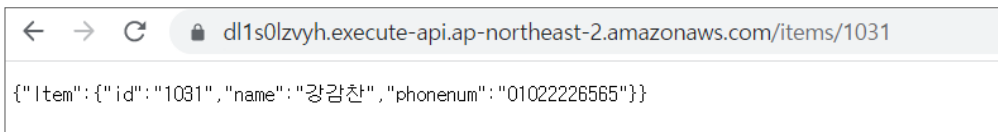
A. 전체목록 호출 GET /items

<https://dl1s0lzvyh.execute-api.ap-northeast-2.amazonaws.com/items>



B. 단일호출 GET /items/{id}

<https://dl1s0lzvyh.execute-api.ap-northeast-2.amazonaws.com/items/1031>

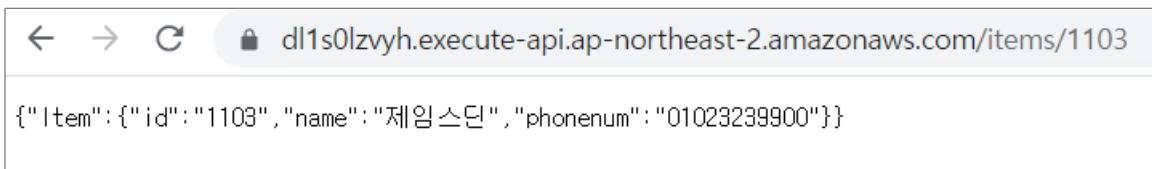


C. 데이터등록 PUT /items

```
curl -X PUT https://dl1s0lzvyh.execute-api.ap-northeast-2.amazonaws.com/items -H "Content-Type: application/json" -d '{ "id": "1103", "name": "제임스딘", "phonenum": "01023239900" }'
```

(출력) "Put item 1104"

등록확인 (GET)



E. 데이터삭제 DELETE /items/{id}

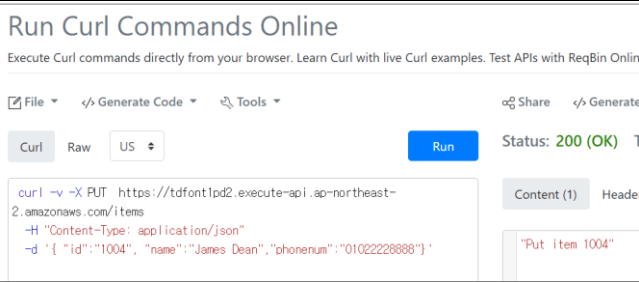
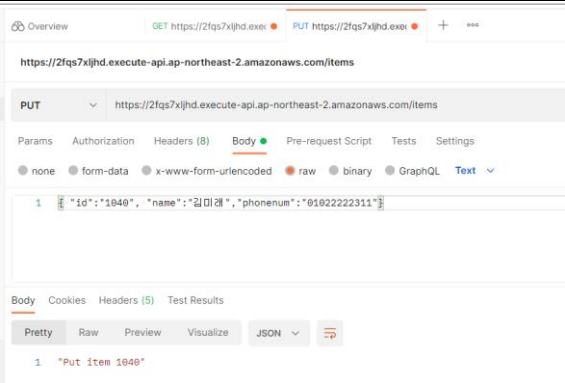
```
curl -X DELETE https://dl1s0lzvyh.execute-api.ap-northeast-2.amazonaws.com/items/1103
```

(출력) "Delete item 1103"

F. API Test 명령어 샘플

(1) PUT 데이터입력

명령	출력
<pre>curl -v -X PUT https://tdfont1pd2.execute-api.ap-northeast-2.amazonaws.com/items -H "Content-Type: application/json" -d '{ "id": "1004", "name": "James Dean", "phonenum": "01022228888" }'</pre>	"Put item 1004"

※ 유의사항 : 경로확인 유의 (예제에서는 /items) (Invoke URL + API Path)	
	
(테스트 - 비회원) https://reqbin.com/curl	(테스트 - 포스트맨 다운로드 혹은 회원가입) https://www.postman.com/

(2) GET 데이터 단항목 가져오기

명령	출력
<pre>curl -v https://tdfont1pd2.execute-api.ap-northeast-2.amazonaws.com/items/1005</pre> ※ Path : /items/{id}	<pre>{ "Item": { "id": "1005", "name": "Tommy Kim", "phonenum": "02050003100" } }</pre>

(3) GET 데이터 전체 가져오기

명령	출력
<pre>curl https://tdfont1pd2.execute-api.ap-northeast-2.amazonaws.com/items</pre> ※ Path : /items	<pre>{ "Items": [{ "id": "1004", "name": "James Dean", "phonenum": "01022228888" }, { "id": "1020", "name": "임격정", "phonenum": "01020003000" }, { "id": "1040", "name": "홍삼정", "phonenum": "01020004000" }, { "id": "1000", "name": "홍길동", "phonenum": "01010001122" }, { "id": "1005", "name": "Tommy Kim", "phonenum": "02050003100" }], "Count": 5, "ScannedCount": 5 }</pre>

(4) DELETE 데이터삭제

명령	출력
<pre>curl -X DELETE curl -v https://tdfont1pd2.execute-api.ap-northeast-2.amazonaws.com/items/1004</pre> ※ Path : /items/{id}	<pre>"Deleted item 1004"</pre>

▶ 오류메시지

Lambda내부 로직오류

```
{"message": "Internal Server Error"}
```

--> CloudWatch에서 로그를 통해서 오류 내용을 확인후 조치 (권한, 문법오류, 연동오류 등)

CORS 오류 : 브라우저 개발자콘솔에서 확인

```
Access to fetch at 'https://i44y7uny7h.execute-api.ap-northeast-2.amazonaws.com/items' from  
origin 'http://app.myservice.com' has been blocked by CORS policy: No 'Access-Control-Allow-  
Origin' header is present on the requested resource. If an opaque response serves your needs, set  
the request's mode to 'no-cors' to fetch the resource with CORS disabled.
```

===== E. 호출테스트 =====

[정적페이지에서 API-Gateway 호출]

버킷생성

- 방식 1 : 콘솔에서 생성 (버킷명을 실습예제에서는 s3://계정번호 12 자리숫자-front-phonebook)
- 방식 2 : CLI 로 생성
aws s3 mb s3://111122223333-front-phonebook

정적페이지 및 권한설정

- 정적웹으로 설정
- Public 허용 및 정책붙임

----- 정책내용 -----

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "S3PublicPart",
      "Effect": "Allow",
      "Principal": "*",
      "Action": [
        "s3:DeleteObject",
        "s3:GetObject",
        "s3:ListBucket",
        "s3:PutObject",
        "s3:PutObjectAcl"
      ],
      "Resource": [
        "arn:aws:s3:::111122223333-front-phonebook",
        "arn:aws:s3:::111122223333-front-phonebook/*"
      ]
    }
  ]
}
```

정적페이지호출 (using JavaScript in S3)

~/apiclass : 디렉토리를 S3로 올려서 정적웹주소로 확인

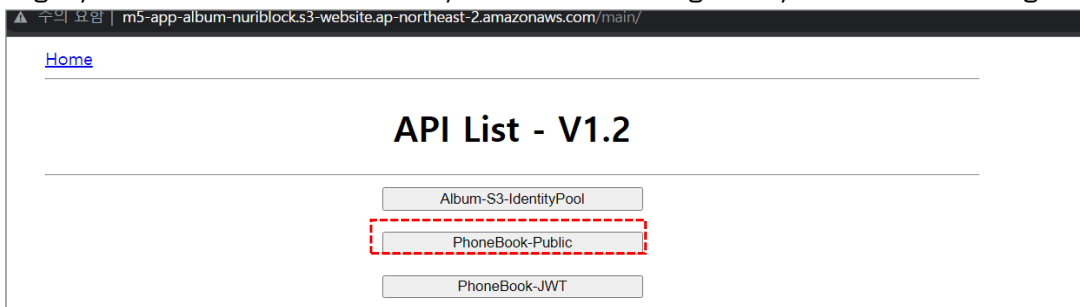
\$ vi ./apistart/main/phonebook/main.js : 아래의 밑줄 API엔드포인트(호출주소) 변경

```
const apiInvokeBaseURL = "https://i1vfadfady2.execute-api.ap-northeast-2.amazonaws.com";
```

(모두 업로드)

\$ cd ./apistart

```
aws s3 cp --recursive . s3://111122223333-front-phonebook --recursive --exclude  
".git/*" --exclude "reference/*" --exclude ".github/*" --exclude ".gitignore"
```



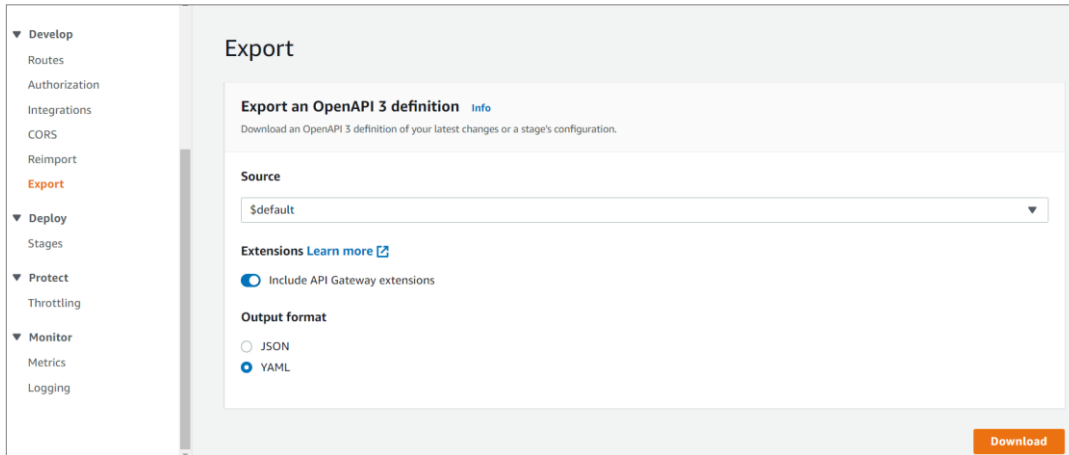
GET /items

전화번호부 목록		
추가		
ID	Name	Phone
1003	미켈란젤로	01022990088
1011	오나라	01010101111
1012	축나라	01020201111
1013	한나라	01030301111

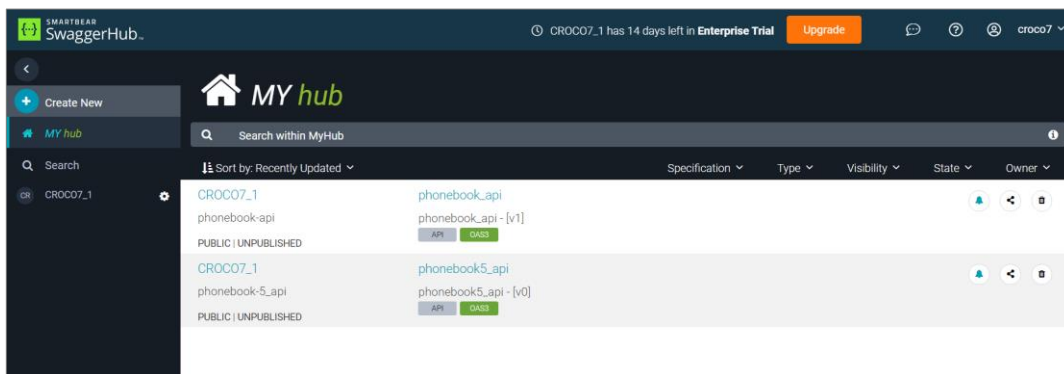
PUT /items	GET /item/{id}
데이터 수정 ID : 1032 Name : 김유신 Phone : 01033337788 확인	전화번호부 Information Id : 1032 Name : 김유신 Phone : 01033337788 수정 삭제

[첨부 1. OpenAPI 3 - SWAGGER 내보내기]

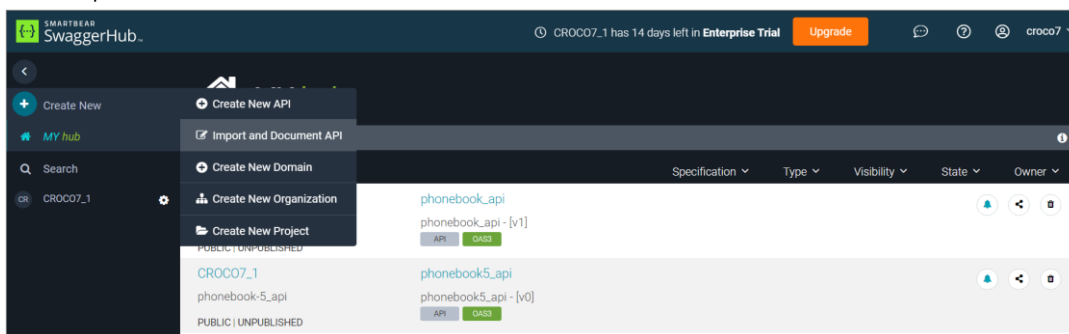
- swagger.io 서비스 이용하거나 swagger 구현
- API 내보내기 (AWS > API-Gateway)

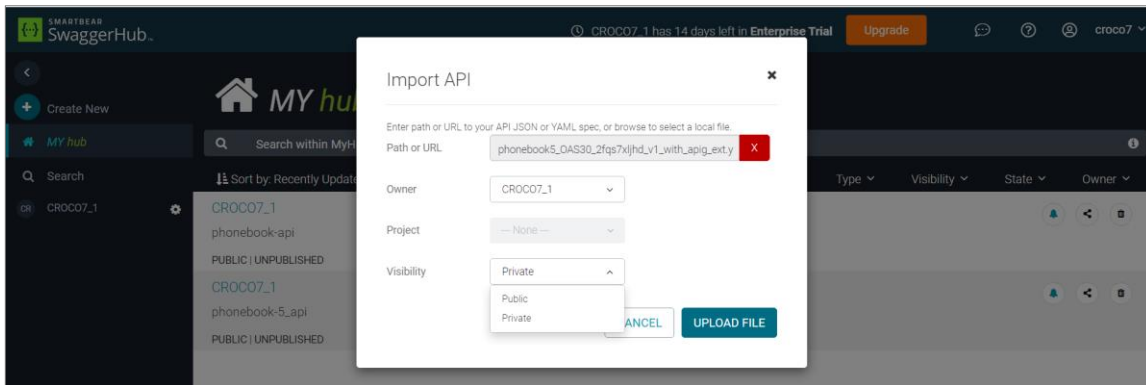


- API 가져오기 (SWAGGER)

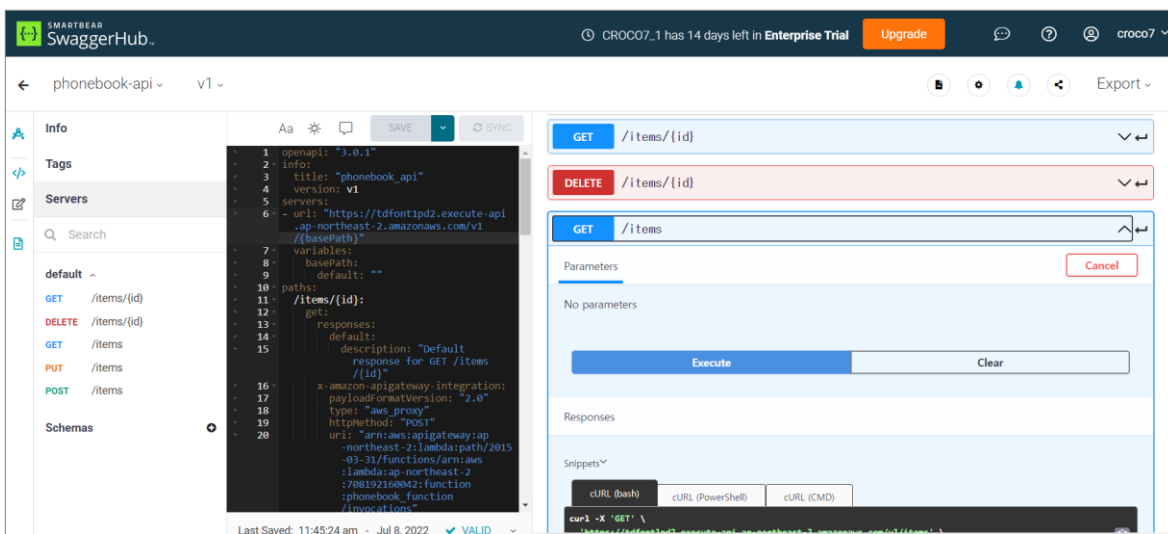


--> Import and Document API





- 실행



[첨부 2. CLI 활용 정책 및 역할추가]

API-Gateway + Lambda + DynamoDB 사용 Role 작성

(cd ~/apistart/reference/iam_ref 에 관련파일 모두 있음)

최종작성 RoleName : simple_apigateway_lambda_role

1) 정책추가

- 로그쓰기 정책 : posvc_log_create_record

(1) Log 기록	(2) Lambda 실행
policy_svc_log_create_record.json	policy_svc_lambda_invoke.json
<pre>{ "Version": "2012-10-17", "Statement": [{ "Effect": "Allow", "Action": ["logs:CreateLogGroup", "logs:CreateLogStream", "logs:PutLogEvents"], "Resource": "*" }] }</pre>	<pre>{ "Version": "2012-10-17", "Statement": [{ "Effect": "Allow", "Action": "lambda:InvokeFunction", "Resource": "*" }] }</pre>
(3) 다이나모 DB	API-Gateway 실행
policy_svc_dynamodb_rw.json	policy_svc_apigateway_invoke.json
<pre>{ "Version": "2012-10-17", "Statement": [{ "Effect": "Allow", "Action": ["dynamodb:Query", "dynamodb:DeleteItem", "dynamodb:GetItem", "dynamodb:PutItem", "dynamodb:Scan", "dynamodb:UpdateItem"], "Resource": "*" }] }</pre>	<pre>{ "Version": "2012-10-17", "Statement": [{ "Effect": "Allow", "Action": ["execute-api:Invoke", "execute-api:ManageConnections"], "Resource": "arn:aws:execute-api:*:*:*" }] }</pre>

※ 비고) 위의 **policy_svc_apigateway_invoke.json** 내용은 AmazonAPIGatewayInvokeFullAccess 권한과 동일함.

(정책생성 CLI)

aws iam create-policy --policy-name my-policy --policy-document file://policy

위의 정책 생성 파일을 모두 CLI 명령어로 실행

aws iam create-policy --policy-name svc_log_create_record --policy-document file://policy_svc_log_create_record.json

aws iam create-policy --policy-name svc_lambda_invoke --policy-document file://policy_svc_lambda_invoke.json

aws iam create-policy --policy-name svc_dynamodb_rw --policy-document file://policy_svc_dynamodb_rw.json

aws iam create-policy --policy-name svc_apigateway_invoke --policy-document file://policy_svc_apigateway_invoke.json

[실행결과]

```
{
  "Policy": {
    "PolicyName": "svc_apigateway_invoke",
    "PolicyId": "ANPA2JY4KZEVM5HIX4NHM",
    "Arn": "arn:aws:iam::708192160042:policy/svc_apigateway_invoke",
    "Path": "/",

```



```

        "DefaultVersionId": "v1",
        "AttachmentCount": 0,
        "PermissionsBoundaryUsageCount": 0,
        "IsAttachable": true,
        "CreateDate": "2022-07-13T01:54:26+00:00",
        "UpdateDate": "2022-07-13T01:54:26+00:00"
    }
}

```

2) 역할(Role)생성 (신뢰관계추가 + 정책붙이기)

- 신뢰관계 정의파일 : **trust_apigateway_lambda.json**

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": [
          "apigateway.amazonaws.com",
          "lambda.amazonaws.com"
        ]
      },
      "Action": "sts:AssumeRole"
    }
  ]
}

```

- 신뢰관계 설정 - Role 생성시 진행

(CLI 명령어)

```
aws iam create-role --role-name example-role --assume-role-policy-document file://example-role-trust-policy.json
```

(실행)

```
aws iam create-role --role-name simple_apigateway_lambda_role --assume-role-policy-document file://trust_apigateway_lambda.json
```

[실행결과]

```

{
  "Role": {
    "Path": "/",
    "RoleName": "simple_apigateway_lambda_role",
    "RoleId": "ARO0A2JY4KZEVDPLPV0C5P",
    "Arn": "arn:aws:iam::708192160042:role/simple_apigateway_lambda_role",
    "CreateDate": "2022-07-13T02:08:20+00:00",
    "AssumeRolePolicyDocument": {
      "Version": "2012-10-17",
      "Statement": [
        {
          "Effect": "Allow",
          "Principal": {
            "Service": [
              "apigateway.amazonaws.com",
              "lambda.amazonaws.com"
            ]
          },
          "Action": "sts:AssumeRole"
        }
      ]
    }
  }
}

```

- 정책추가

(정책추가 CLI 명령어)

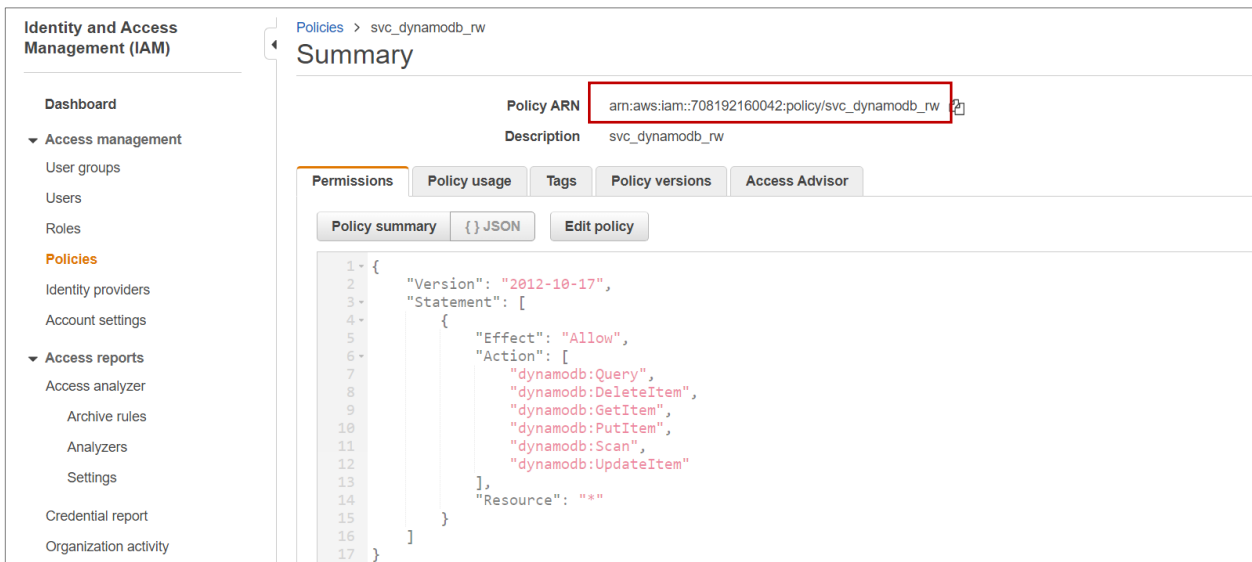
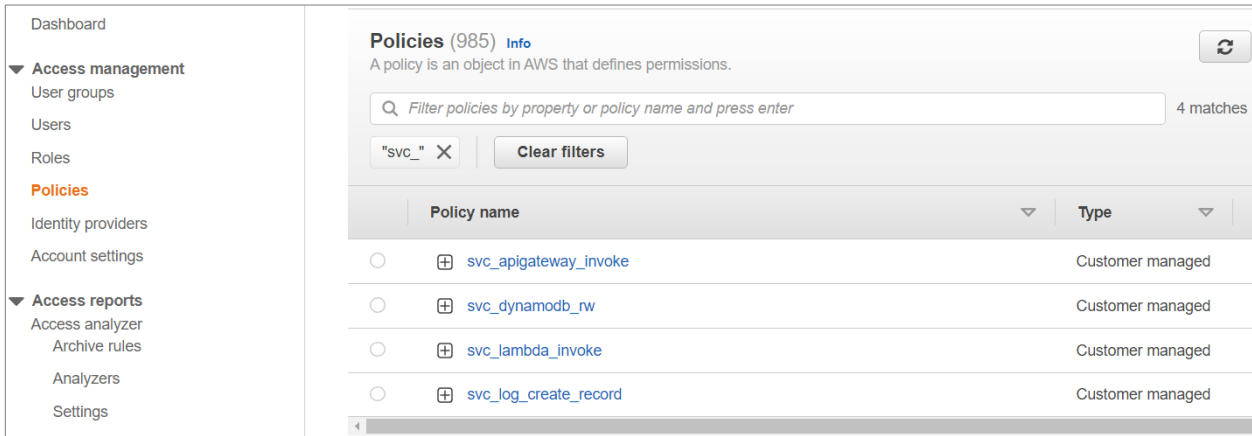
```
aws iam attach-role-policy --role-name example-role --policy-arn "arn:aws:iam::aws:policy/AmazonRDSReadOnlyAccess"
```

(정책확인 CLI 명령어)

```
aws iam list-attached-role-policies --role-name example-role
```

- 정책추가 진행

Console > IAM > Policies > 필터링 "svc_" : 정책의 ARN 을 확인할 수 있음.



(명령어 생성) 111122223333 : AWS 계정번호에 해당 (각자 변경)

```
aws iam attach-role-policy --role-name simple_apigateway_lambda_role --policy-arn arn:aws:iam::111122223333:policy/svc_apigateway_invoke
```

```
aws iam attach-role-policy --role-name simple_apigateway_lambda_role --policy-arn arn:aws:iam::111122223333:policy/svc_dynamodb_rw
```

```
aws iam attach-role-policy --role-name simple_apigateway_lambda_role --policy-arn arn:aws:iam::111122223333:policy/svc_lambda_invoke
```

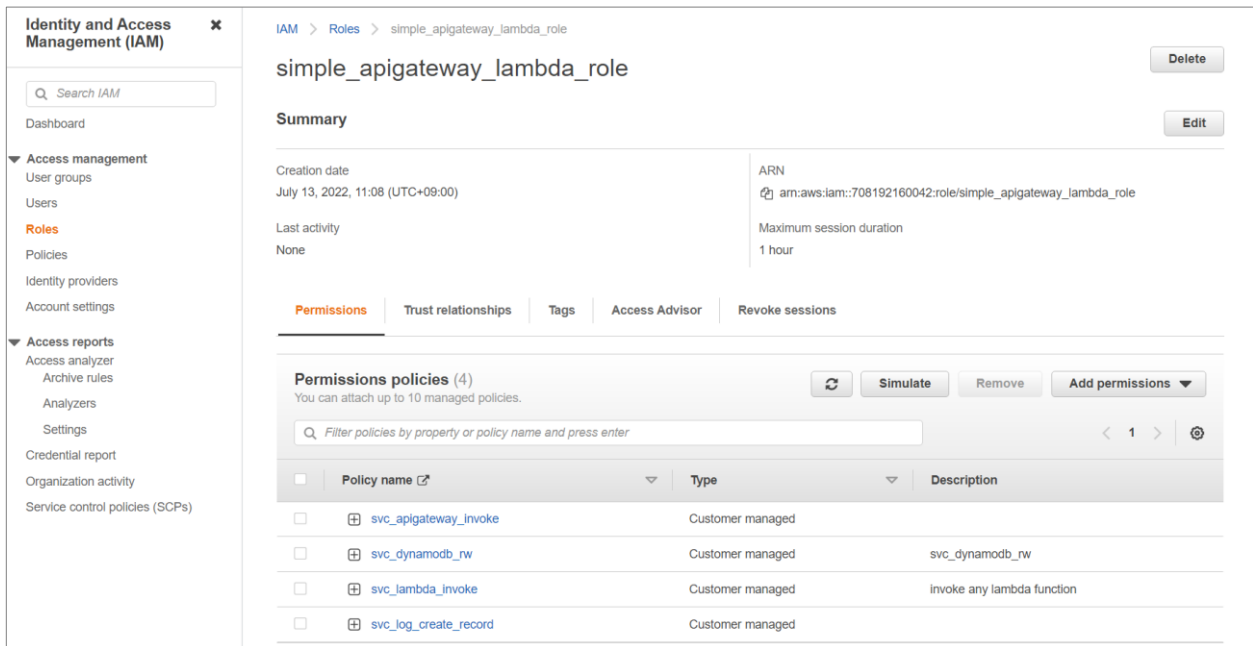
```
aws iam attach-role-policy --role-name simple_apigateway_lambda_role --policy-arn arn:aws:iam::111122223333:policy/svc_log_create_record
```

- 최종적으로 추가된 역할(Role) simple_apigateway_lambda_role 을 확인함 .

권한(Permissions)와 신뢰관계(Trust relationships) 확인

Role 의 ARN 은 자주 사용됨 - CLI 템플릿 및 명령어

여제) `arn:aws:iam::708192160042:role/simple_apigateway_lambda_role`



The screenshot shows the AWS IAM console interface for the role 'simple_apigateway_lambda_role'. The left sidebar contains navigation links for 'Access management' (User groups, Users, Roles, Policies, Identity providers, Account settings) and 'Access reports' (Access analyzer, Archive rules, Analyzers, Settings, Credential report, Organization activity, Service control policies (SCPs)). The main content area shows the role's summary and permissions. The 'Permissions' tab is selected, displaying a list of four policies: 'svc_apigateway_invoke', 'svc_dynamodb_rw', 'svc_lambda_invoke', and 'svc_log_create_record'. Each policy is marked as 'Customer managed'.

Policy name	Type	Description
svc_apigateway_invoke	Customer managed	
svc_dynamodb_rw	Customer managed	svc_dynamodb_rw
svc_lambda_invoke	Customer managed	Invoke any lambda function
svc_log_create_record	Customer managed	

[Lambda 실행기본 역할 : Authorizer 등에서 사용]

1) 역할생성

```
aws iam create-role --role-name simple_lambda_role --assume-role-policy-document  
file://trust_lambda.json
```

2) 정책붙이기

```
aws iam attach-role-policy --role-name simple_lambda_role --policy-arn  
arn:aws:iam::111122223333:policy/svc_lambda_invoke  
  
aws iam attach-role-policy --role-name simple_lambda_role --policy-arn  
arn:aws:iam::111122223333:policy/svc_log_create_record
```

[첨부 3. 사용자원 삭제]

- lambda 삭제

[node]

```
aws lambda delete-function --function-name func_phonebook_node
```

```
aws lambda delete-function --function-name func_phonebook2_node
```

[python]

```
aws lambda delete-function --function-name func_phonebook_python
```

```
aws lambda delete-function --function-name func_phonebook2_python
```

- s3 삭제

```
aws s3 rm s3://111122223333-front-phonebook --recursive
```

```
aws s3 rb s3://111122223333-front-phonebook
```

- dynamodb 삭제

```
aws dynamodb delete-table --table-name phonebook
```

- apigateway 삭제 (콘솔에서 api-id 확인후 진행)

```
aws apigatewayv2 delete-api --api-id abcd111e
```

- 참조소스 삭제

```
rm -rf ~/apistart
```

- 끝 -